

CLOUD NATIVE
COMPUTING FOUNDATION

Kelpflux

Elastic Slurm scheduling on Kubernetes
for shared GPU AI workloads.

Agenda

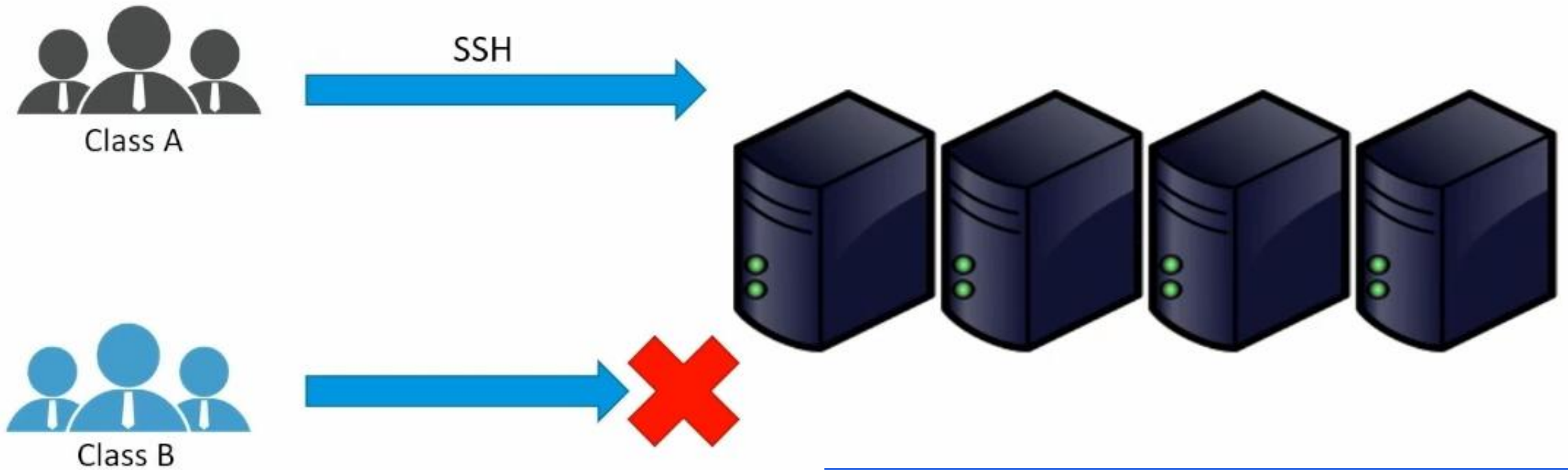
-  **1 Introduction**
-  **2 System Architecture**
-  **3 Demo & Evaluation**
-  **4 References**



01. Introduction

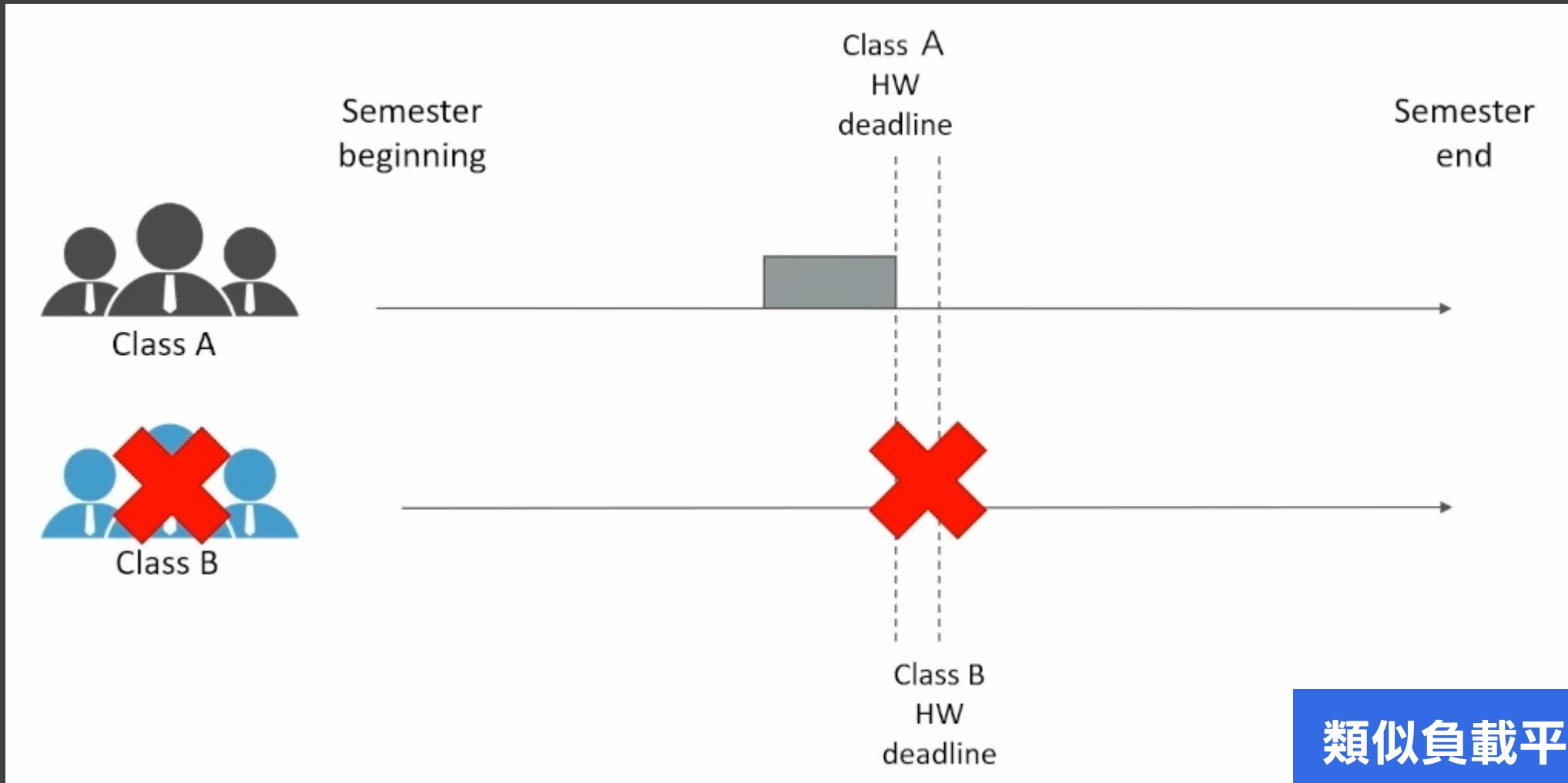
專題簡介

Example Scenarios



有沒有辦法多台機器給多個人使用？

Example Scenarios



類似負載平衡的概念

Slurm

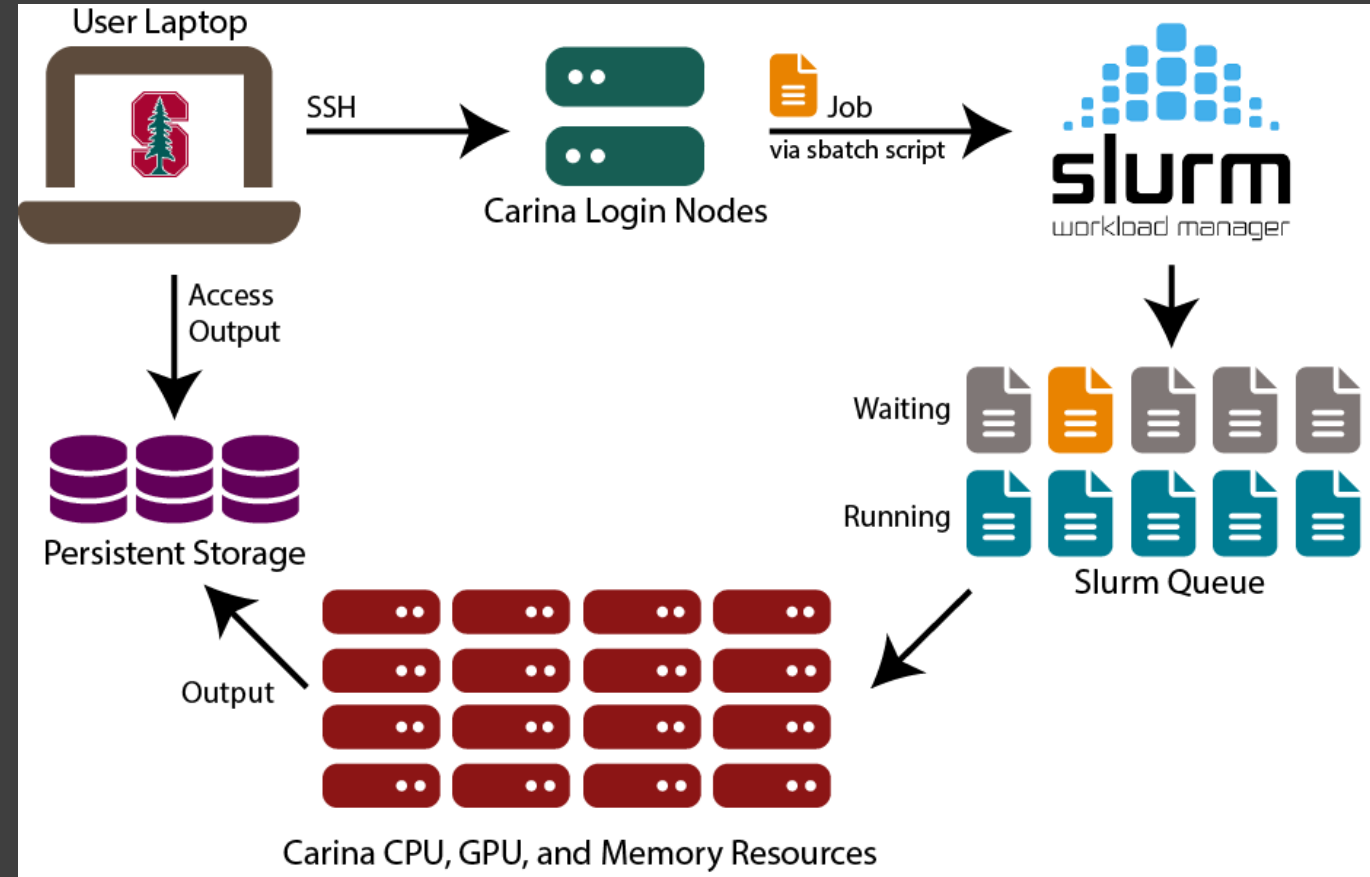
What is Slurm ?

- Slurm = Simple Linux Utility for Resource Management
- An open source system
 - 管理工作：資料分配、排程、執行



Roles in Slurm

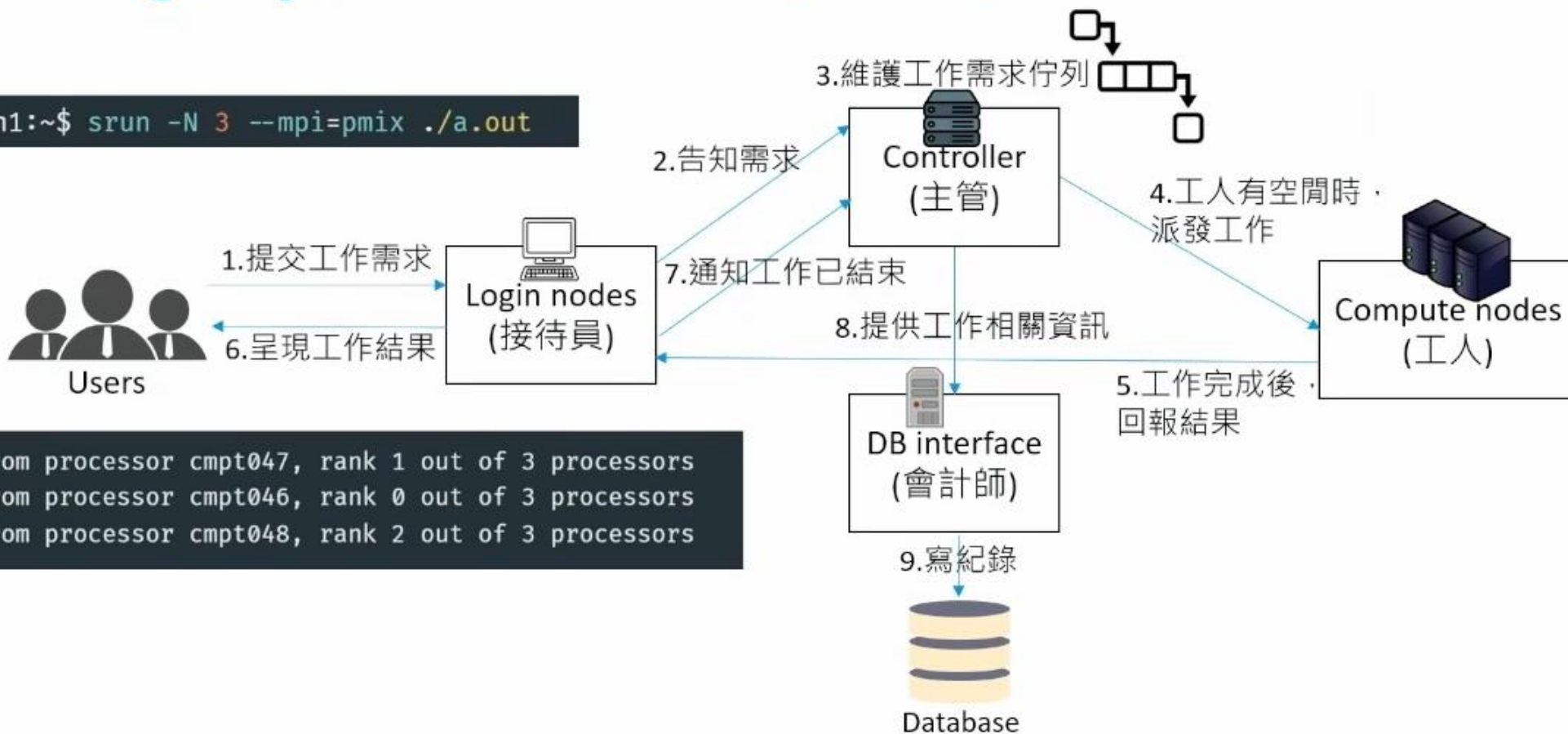
- Login nodes = 接待員/客服
- Controller = 主管
- Compute nodes = 工人
- DB interface = 會計師



Running a job in Slurm (srun)

Running a job in Slurm (srun)

```
bjhuang@login1:~$ srun -N 3 --mpi=pmix ./a.out
```



```
Hello world from processor cmpt047, rank 1 out of 3 processors  
Hello world from processor cmpt046, rank 0 out of 3 processors  
Hello world from processor cmpt048, rank 2 out of 3 processors
```

Kubernetes

What is Kubernetes ?

- Kubernetes (K8s) = Containers Orchestration Tool
- An open source system
 - 管理容器：啟動、運算資源、排程、自動部署
 - 大公司架設服務很愛用

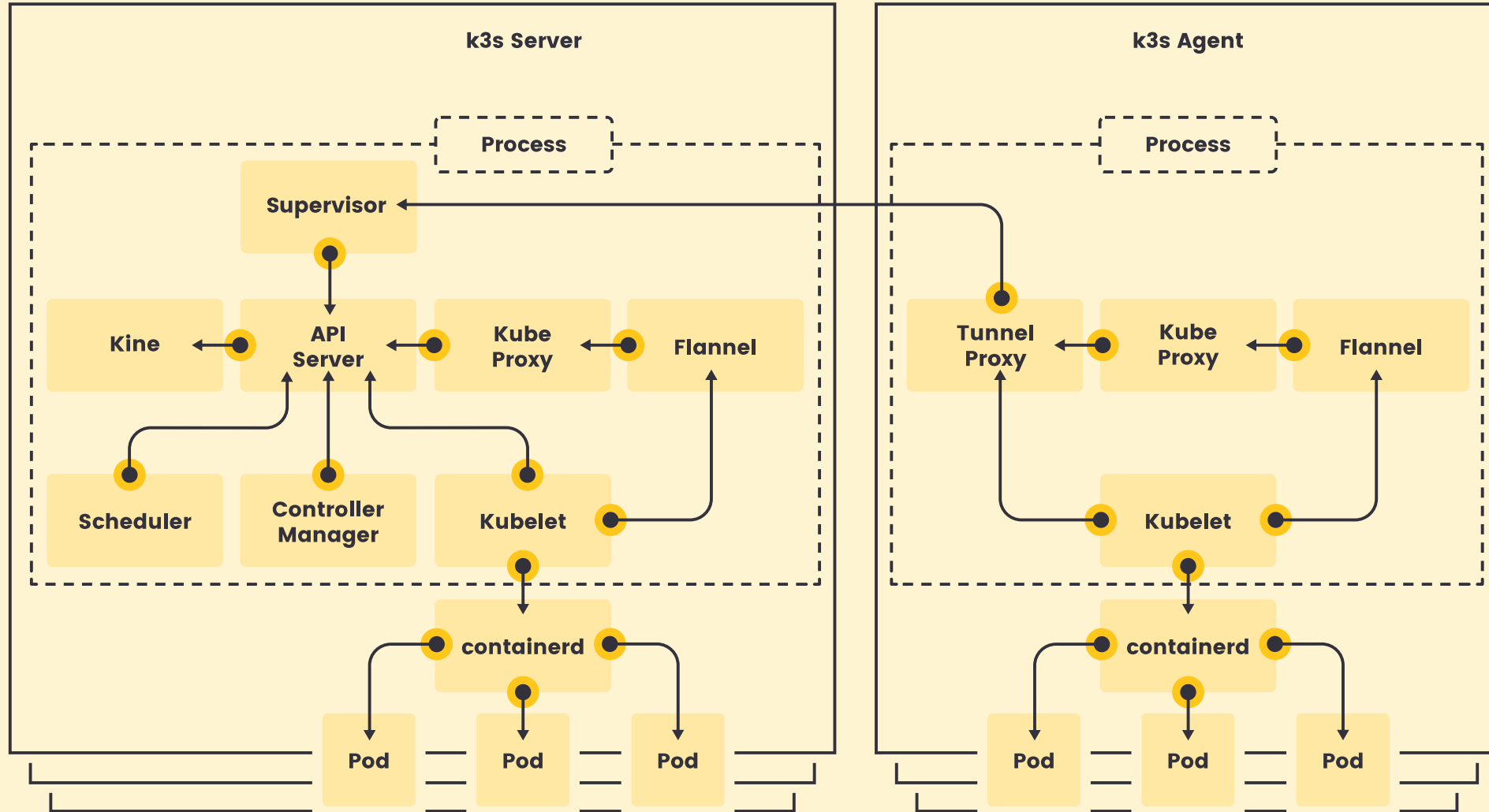


Kubernetes Distribution

- 本來是用 kind，但硬體支援度有極限，改成使用 K3s
- 建立在 Kubernetes 標準安裝之上的一個輕量發行版
- 由 Rancher Labs 開發，通過 CNCF 認證。



K3s



Background

Motivation

- Kubernetes 很擅長資源編排與彈性伸縮，但對 HPC / AI 訓練工作負載的語意支援有限。
 - Slurm 很擅長批次排程、資源配額與叢集治理，但傳統部署通常偏靜態，對雲端環境的節點生命週期不夠友善。
- ⇒ 打造一個讓多位使用者共用的批次 AI 工作平台



Benefits

- 提升資源利用率
 - Slurm 可以透過將工作排程到 K8s 叢集中的可用資源
- 簡化的工作負載管理
 - 一律透過 Slurm 介面，用於管理 ML 工作負載
- 提升可擴展性
 - Slurm 可以透過將機器學習工作負載排程到多個 K8s 叢集來擴展

Slurm on Kubernetes

- Slurm 決定誰用資源、什麼時候用、用多少
 - 計算資源排程問題
- Kubernetes 決定資源怎麼存在、怎麼啟動、怎麼管理
 - 動態提供 Slurm 需要的運算節點
 - 當硬體調整的時候不用重新調整 Slurm 設定
 - 依賴版本管理、網路設定

Our Feature !



- 擴容、擴縮、低功耗、節點壞掉重啟
- 客製化排程演算法、運算資源分配
- 支援多種 Job
- 近年來企業很重視安全，可以裝個龍蝦



Reliability



Performance



Lifecycle Unity



Observability



02. System Architecture

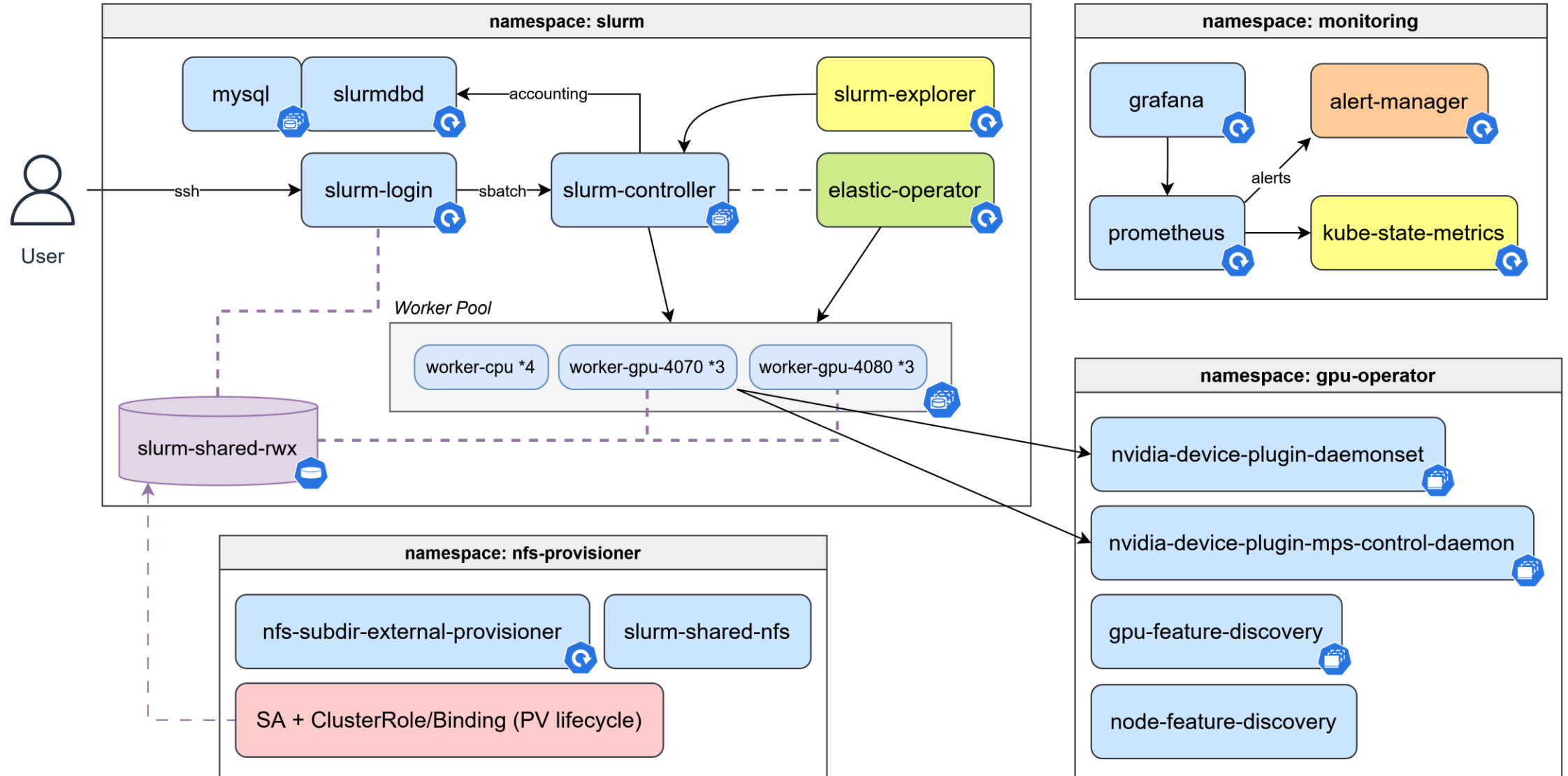
系統設計與架構

Development Environment



- Ubuntu 24.04 + K3s v1.34 + RTX 4070 (lab)
- Operator : Python 3.11-slim + kubectl CLI
- Nvidia GPU Operator

K8s (K3s) Cluster



Slurm GRES

- Slurm 的通用資源管理機制，讓 Slurm 能夠追蹤和排程 CPU/Memory 以外的資源 (e.g. GPU)
- 知道每個 node 上有哪些 GPU (型號、數量)
- 讓使用者提交 job 時指定需要的 GPU 類型和數量
- 確保 GPU 資源不會被過度分配

GPU Resources

Multi-Process Service (MPS)

- 在 GPU 上創建多個 context，將 GPU 的資源分配給多個 process
- 每個進程擁有一個獨立的 context，可以獨立執行其計算任務
- 提高多容器共享 GPU 的 QoS
- 不管 VRAM、記憶體頻寬、L2 Cache、編解碼器、Copy Engines
 - export `CUDA_MPS_PINNED_DEVICE_MEM_LIMIT=0,3072` (slot_0 3072 MB)

NVIDIA GPU MPS



請求 + constraint	RTX 4070 分配 SM	RTX 4080 分配 SM	適合工作
<code>--gres=mps:50 --constraint=gpu-rtx4070</code>	~24 SM (50%)	—	中型推論 (7B LLM serving)
<code>--gres=mps:25 --constraint=gpu-rtx4070</code>	~12 SM (25%)	—	小型推論 (image classifier)
<code>--gres=gpu:rtx4070:1</code>	48 SM (整卡)	—	訓練 (≤12 GB VRAM)
<code>--gres=gpu:rtx4080:1</code>	—	76 SM (整卡)	訓練 (≤16 GB VRAM)
<code>-N 2 --gres=gpu:1</code>	48 SM	76 SM	2-GPU DDP

Job Migration

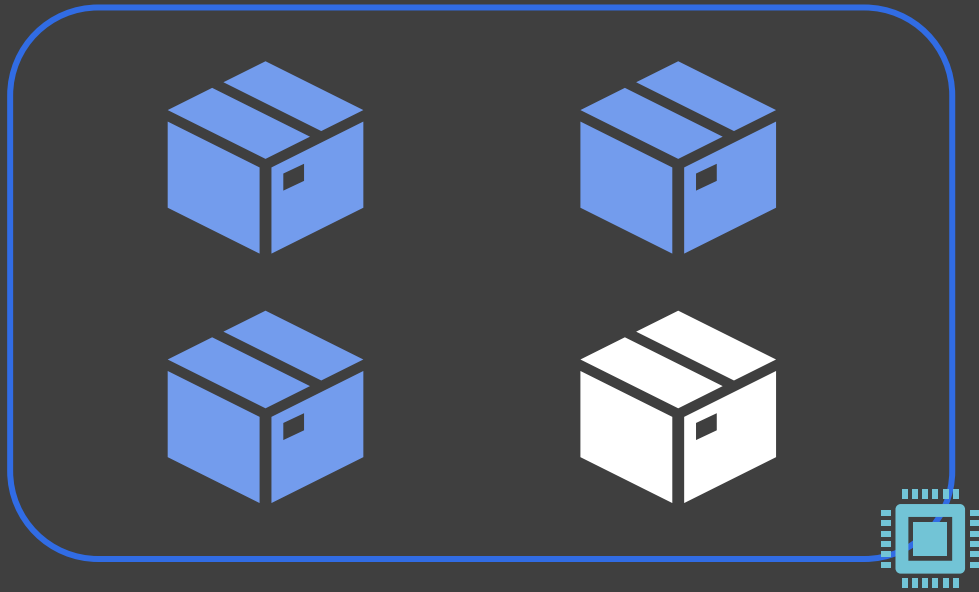
Job A



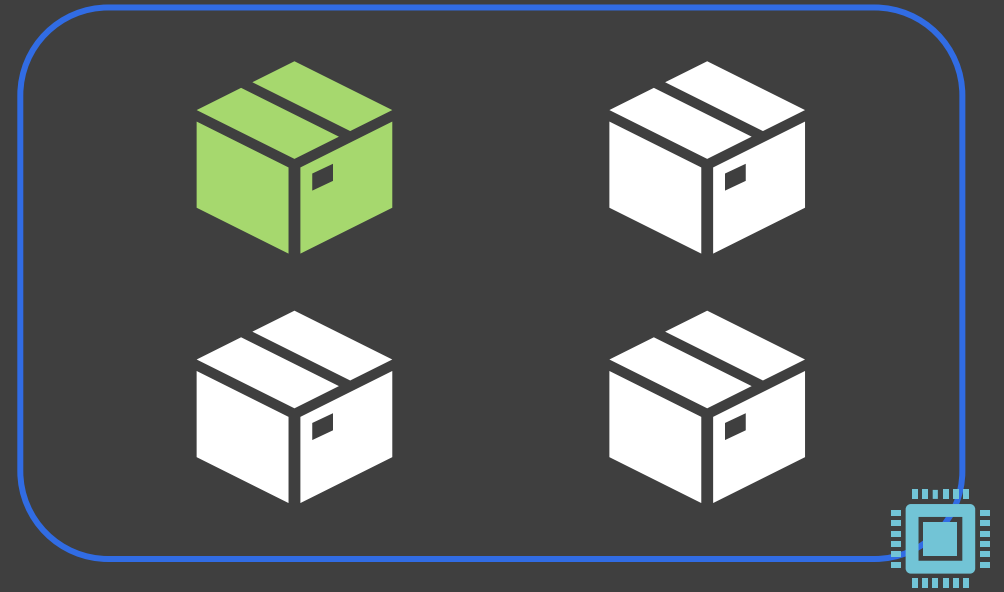
Job B



Job C

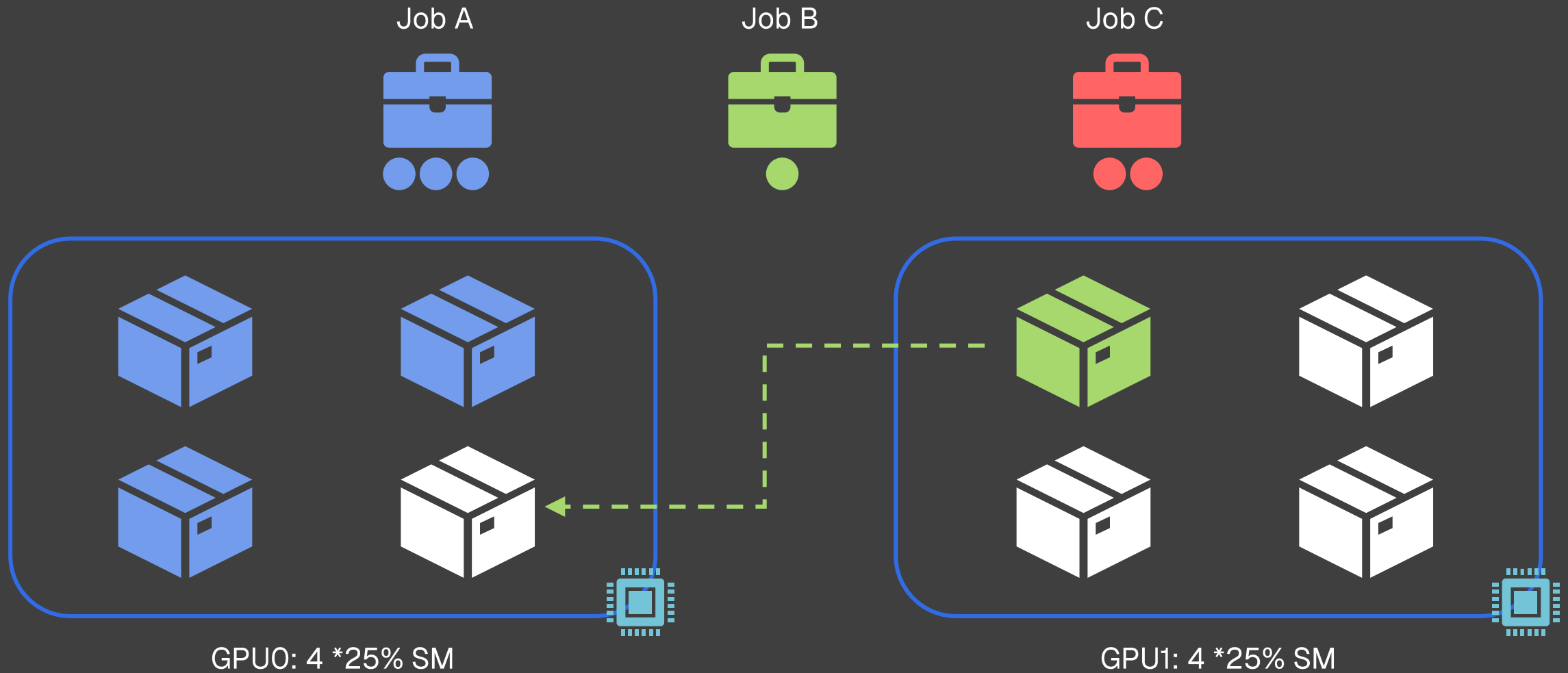


GPU0: 4 * 25% SM



GPU1: 4 * 25% SM

Job Migration



Job Migration

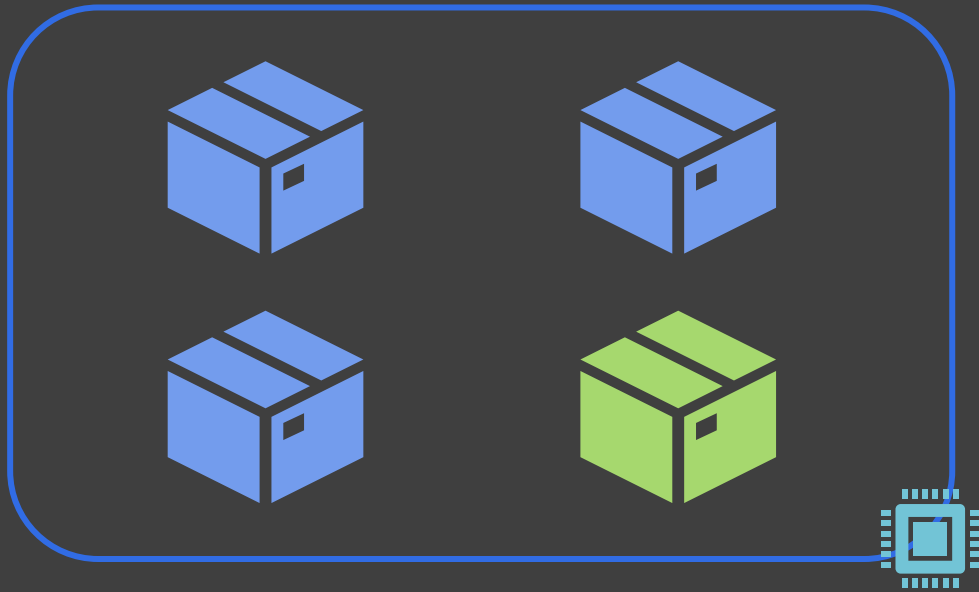
Job A



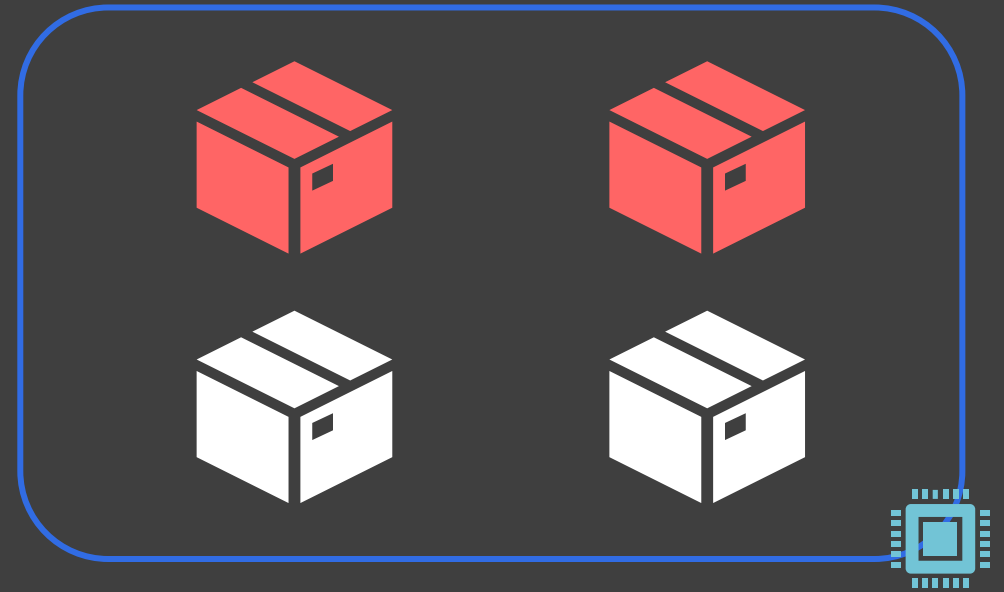
Job B



Job C



GPU0: 4 * 25% SM



GPU1: 4 * 25% SM

Job Scheduling / Migration

Slurm Scheduling Strategy (1/2)

- Bin-packing：先塞滿第一張卡，再用第二張卡
- 預設情況下，Slurm 為了散熱和負載平衡，傾向於把 Job 分散到不同的節點。

```
slurm:
  config: |
    SelectType=select/cons_tres
    # 重點：CR_Core_Memory 配合 CR_Pack_Nodes
    # 這會讓 Slurm 優先把任務填入同一個資源塊
    SelectTypeParameters=CR_Core_Memory,CR_Pack_Nodes
```

當 Job B 進來要求 25% MPS 時，Slurm 會檢查 GPU0。如果 GPU0 已經有 Job A (75%)，加起來剛好 100%，它會優先把 Job B 塞進 GPU0。GPU1 就會保持 100% 空閒，留給未來的 Job C。

Slurm Scheduling Strategy (2/2)

- Preemption：需要定義不同的 Partition 優先級
 - HighPrio Partition: 給 Job C (重要大任務)，優先級 10。
 - LowPrio Partition: 給 Job B (小任務)，優先級 1。
- 當 Job C 提交到 HighPrio 時，Slurm 會自動把 GPU1 上的 Job B 踢掉，讓 Job C 執行
- 等到 GPU0 或 GPU1 有空位時，Job B 會自動重新啟動

```
slurm:
  config: |
    PreemptMode=REQUEUE           # 或者用 SUSPEND (如果 Job 支援)
    PreemptType=preempt/partition_prio
    JobRequeue=1
```

Scheduling Strategy Design

- 啟發式排程：改 Slurm 設定即可，快速有效
- 數學式排程：尋找最佳解
 - Analytical modeling + optimization
 - 深度強化學習 (DRL)

Analytical modeling + Optimization

- Score Function：給每個 (job, placement) 算分，挑最高的
- 係數 $\alpha, \beta, \gamma, \delta, \varepsilon$ 怎麼定？
 - 線上學習：把 $\alpha - \varepsilon$ 當可學習的權重，用 LinUCB 等演算法線上更新。
 - 深度強化學習 DRL

Score Function (baseline)

$$\begin{aligned} \text{score}(J, P) = & \alpha \times \text{mps_fit}(J, P) \rightarrow \text{MPS slot 容量配適} \\ & + \beta \times \text{vram_fit}(J, P) \rightarrow \text{VRAM 餘裕} \\ & + \gamma \times \text{topology}(J, P) \rightarrow \text{NCCL 拓撲親和性} \\ & - \delta \times \text{fragmentation}(J, P) \rightarrow \text{放完後碎片化代價} \\ & + \epsilon \times \text{pred_runtime}(J) \rightarrow \text{預測 runtime} \end{aligned}$$

$$\alpha = 0.40, \beta = 0.20, \gamma = 0.0, \delta = 0.20, \epsilon = 0.0$$

Runtime Predictor (M6)

1. 在現有 Slurm + GPU C
2. 用 Slurm plugin 把 sco
3. 在實機跑 10~50 個 job
4. 把打分函數的 $\alpha - \varepsilon$ 當
5. PPO + 公開 trace 訓練
6. 對比公式 + RL 與純公式

FastAPI + LightGBM 服務預測 job 執行時間，Lua plugin 通過 HTTP 調用：

`scheduler-score-spec.md:79-84`

- **Features:** user_name, gpu_count, mps_req, gpu_type, hour_of_week, user_freq, user_mean_log_rt
- **Timeout:** 200ms 避免阻塞 slurmctld
- **Fallback:** 服務不可用時回傳 0.5 (中性分數)

Lua 調用範例：`configmap-job-submit.yaml:69-79`

```
local PRED_ENABLED = true
local PRED_URL = "http://runtime-predictor:8080/predict"
local PRED_TIMEOUT_S = 200 / 1000.0
```

Weight Tuner (M9)

- 使用 UCB1 bandit 演算法離線優化權重組合：

UCB1 選出的最佳 arm 參數為：

參數	因子名稱	數值	說明
α	MPS fit	0.10	降低 MPS 配適權重
β	VRAM fit	0.20	固定值
γ	Topology	0	未實作
δ	Fragmentation cost	0.05	降低碎片化懲罰
ϵ	Predicted runtime	0.60	大幅提高 runtime predictor 權重

UCB1 Bandit (RL)

- 選擇「目前看起來最好的」，還是去嘗試「那些還不熟悉的」？
- 為每一個選項計算一個分數，每次都選擇分數最高的那個

$$\text{UCB}_i = \bar{x}_i + \sqrt{\frac{2 \ln N}{n_i}}$$

該選項過去的
平均獎勵

N 是總次數， n_i 是該
選項被選中的次數。

Scheduler Summary (1/3)

Deep RL

- Shadow mode , 因為目前 policy 還不可靠

UCB1 Weight tuner

- 每 300 秒查看工作歷程
- 用 bandit 自動調整 $(\alpha, \delta, \varepsilon)$ 參數
- Lua plugin 載入時覆蓋係數

Score + Slurm Settings

- 每次 sbatch 時計算分數並寫入 priority
- f_pred_runtime 呼叫 LightGBM 預測器服務
- Slurm backfill 排程器用 priority 來排序工作

Scheduler Summary (2/3)

$$\text{priority} = \alpha \times f_{mps_fit} + \beta \times f_{vram_fit} + \gamma \times f_{topology} \\ - \delta \times f_{fragmentation} + \epsilon \times f_{pred_runtime}$$

- job 需要的 MPS slot vs node 剩餘 slot 的配適度 → 讓小 job 填充空隙
- VRAM 需求 vs. node 等級 → 不浪費高規 GPU
- ~~NCCL 拓撲親和性 (x) → 預留多 GPU job~~
- 接受後 cluster free-MPS 的 stddev (懲罰項) → 不製造碎片
- $\text{clamp}(0, 1 - \text{pred_s} / 14400)$ → 短 job 優先

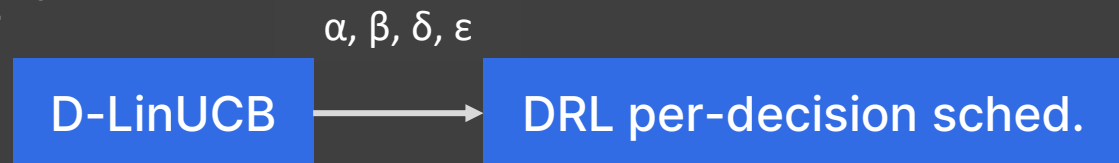
Scheduler Summary (3/3)

- 公式候選參數： $\alpha = 0.10, \beta = 0.20, \delta = 0.05, \varepsilon = 0.30$
- 真正有效的組合是 M5 predictor + M6 wiring + M9 UCB1 weight-tuner。
- M3 score 本身效益不顯著，但是 predictor 接進來的載體。M7 在現有 victim-selection 設計下是 net negative，M10 policy 還在訓練。

M	加入	效果 (sim paired CI)
M1	multifactor + backfill 旋鈕	FCFS→multifactor -72% mean JCT
M3	score 公式 (mps_fit, vram_fit, frag)	vs multifactor $\approx 0\%$ (純 score 沒顯著)
M5+M6	predictor + f_pred_runtime 接線	vs multifactor -20% (philly), -29% (burst)
M7	fragmentation requeue (shadow mode)	+33% 反效果，保持 shadowMode=true
M9	UCB1 bandit 動態調 arm	用 1/3 預算找到接近 grid-best 的權重
M10	PPO RL scheduler (shadow)	policy 仍輸 score，100% abstain

DRL Scheduler

- 直接在 live cluster 跑 online DRL 在小叢集不可行，因此設計：
 1. 模擬訓練：PPO 跑 $10^5 \sim 10^6$ episode 拿到 policy
 2. 實戰微調：用 RLPD (Ball et al., 2023) + 模擬彈道 + 線上彈道來做 fine-tune。
 3. 整合：外層 D-LinUCB (Russac, Y. et al.) 在小時尺度調超參數，內層 DRL 在 per-decision 尺度排程。



Observability

Observability



雲端服務崛起



錯誤排查難度上升

Observability Aspects

Metrics



負責監控系統有什麼狀況，當要發生服務故障前可以透過設定閾值搭配告警提早得知。

Logs

當問題發生時，可以用來查看故障時正在執行哪些服務，以及產生的錯誤資訊。

Traces

在連續的時間維度上，透過 Trace 以及 Span 關聯，把空間給排列展示出來，並且有 Trace-Context 規範，能夠直觀的看到請求在分散式系統中經過所有服務的紀錄。



- 最受歡迎的開源監控與告警工具組，是 CNCF 的畢業專案
- 專為動態雲端環境設計，透過 HTTP 拉取 metrics，儲存於時間序列資料庫中，並使用 PromQL 進行數據查詢分析。
- 負責發 Alert 到 Slack/Telegram



- 能夠將各種資料做成監控儀表板，支援多種資料來源，以及提供豐富的面板選擇，協助我們將資料視覺化。
- Prometheus 預設的 GUI 功能不足，需要 Grafana 這種支援豐富圖表的工具幫助我們可視化 Metrics 資訊。



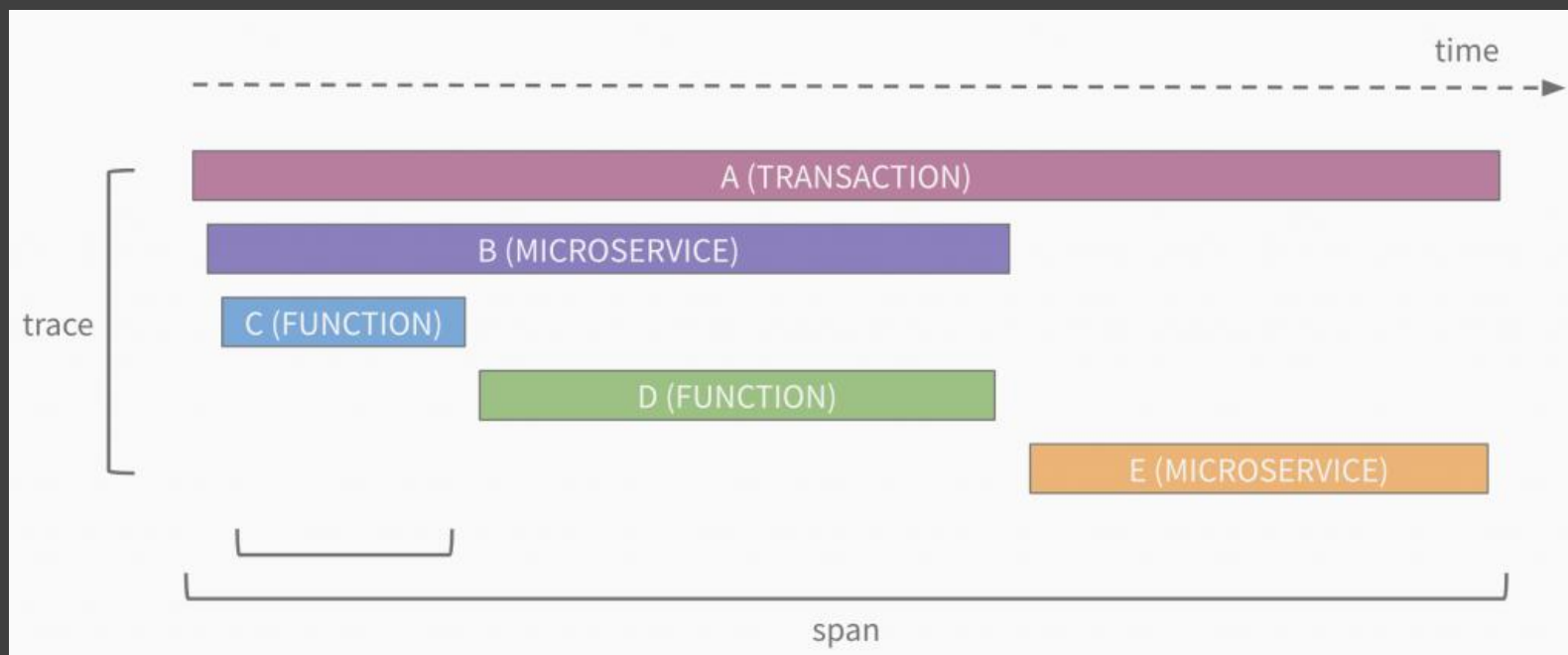
- 由 CNCF 孵化的開源專案，是 Cloud Native 的可觀測性框架
- 提供標準化的 API、SDK 與協議自動檢測、蒐集、導出遙測數據資料 (Metrics、Log、Trace)，並支援 W3C 定義的 Http trace-context 規範

Future Work

Distributed Tracing (1/2)

- 搜集應用程式內部與端到端請求相關的資料，以實現對整個系統的監控和分析。

- 服務名稱
- 操作名稱
- 持續時間



Distributed Tracing (2/2)

- 讓一個 Job 的完整生命週期變成一條可視化的 Trace :

```
[sbatch submit] → [pending in queue] → [scale-up decision]  
→ [K8s provisioning] → [slurmd registration] → [DDP torchrun]  
→ [checkpoint write] → [scale-down decision] → [job complete]
```

- 紀錄 job_id, pool, checkpoint_age, time 等屬性

(+) Tools : OpenTelemetry + Grafana Tempo